# Raport weryfikacji

Naucz Pilota 1.0.0, 3 listopada 2026. Środowisko: macOS arm64, Node 24.11.0, npm 11.6.1; Chromium Playwright. Jest to raport techniczny lokalnego wykonania, nie badanie skuteczności nauczania.

## Wyniki wykonane

- Silnik TypeScript: 24 rozwiązania witness dają SUCCESS; 37 knownFailures odrzucanych dokładnie oczekiwanym kodem. Przeniesiono 42 kontrole parsera i geometrii z walidatora Python, w tym dokładne skracanie dziesiętne, limity, cienką przeszkodę i wewnętrzny wierzchołek.

- Testy jednostkowe/integracyjne: 158 PASS w siedmiu plikach Vitest. Obejmują silnik, maszynę stanów, 32 deterministyczne scenariusze INT z ai-evals i oficjalny SDK na lokalnej atrapie HTTP. Dodano regresję błędu zaobserwowanego w SEM05: schemat dostawcy nie pozwala wskazać reguły bez dowodu ani nieistniejących faktów.

- Python: PASS, 6 misji / 24 sceny / 24 witness / 37 błędów / 42 kontrole. Niezależny od TypeScript walidator referencyjny potwierdza dane, nie zastępuje testu aplikacji.

- Playwright: 7 PASS. Pełna kampania demo z 24 scenami; porażka robota w M04 nie blokuje gracza. Pierwszy wynik jest zachowany po odświeżeniu, poprawkach i powtórce. Przed human_check brak toru, robota, dziennika, wskazówek, obliczonych y i rozmowy.

- Interfejs: widoki 1366x768, 1440x900, 390x844; axe WCAG A/AA bez wykrytych naruszeń na starcie i practice; brak poziomego przewijania, klawiaturowa M01, tekst 200%, reduced motion i demo po wyłączeniu sieci. Zrzuty obejrzano wzrokowo. To ograniczony zakres, bez deklaracji pełnej dostępności dla wszystkich użytkowników.

- AI bez płatnego API: walidacja requestów i faz, ponowne obliczanie dowodów, odmowa, incomplete, deadline, maksymalnie jedna naprawa techniczna, jawna błędna propozycja, idempotencja, konflikt body/revision, limity, brak ukrytego demo i brak kontekstu przyszłych scen. Trwały licznik SQLite sprawdzony po zamknięciu i ponownym otwarciu bazy.

- Oficjalny SDK: testy HTTP completed, incomplete, refusal i 429; strict text.format, store:false, brak narzędzi i wyłączone automatyczne retry. To lokalna atrapa protokołu, nie prawdziwy model.

- Produkcyjny build: frontend bez source maps i bez danych witness/knownFailures. Statyczne walidatory zamiast runtime kompilacji Ajv. Serwer bundluje współdzielone pakiety do JavaScript.

- Electron macOS: cała M01 z lokalnych zasobów app://, bez backendu; izolacja kontekstu, sandbox, brak require/process w rendererze; rzeczywiste zamknięcie i wznowienie z zachowaniem błędnej pierwszej próby po korekcie; PASS w checks/desktop-smoke.json.

- Windows x64: utworzono ZIP zawierający EXE PE i zasoby. Inspekcja ASAR wyklucza node_modules, .env, mapy źródeł i dane autora. Paczka niepodpisana. Cross-build jest osobną czynnością od testu uruchomienia.

- Nagrano rzeczywiste automatyczne przebiegi M04: demo (37,56 s) oraz live (45,28 s), MP4 1440x900 z napisami. Nagranie live wykonało 2 prawdziwe żądania, pokazało oryginalną propozycję robota i zachowało pierwszą nieudaną próbę człowieka po korekcie i odświeżeniu. Nie udaje pilotażu z użytkownikami.

## Uczciwe ograniczenia

Prawdziwe AI: WYKONANO lokalnie. Prompt 1.0.2, model gpt-5.4-mini-2026-03-17, 22 tury i 22 żądania dostawcy w końcowym przebiegu. Wszystkie odpowiedzi przeszły kontrolę struktury, fazy, potwierdzonej pamięci i niezależnego wyniku silnika. Opóźnienia: 1069-2330 ms, mediana 1426 ms; 49505 tokenów wejścia i 2595 wyjścia. To obserwacja tego przebiegu, bez gwarancji podobnych wyników w przyszłości. Wcześniejsze błędy i przerwane próby zachowano osobno w checks/.

Jakość dialogu: wstępny przegląd asystenta jest zapisany w checks/ai-semantic-review.json; akceptacja przez człowieka PENDING. SEM03 otrzymał sound zamiast oczekiwanego partial, a SEM09 partial zamiast incorrect. Piko czasem proponuje objaśnianie lub liczenie za gracza. Komentarz i etykieta modelu nie odblokowują misji. W sześciu końcowych transferach robot zaliczył 2 plansze; 4 propozycje silnik odrzucił. Porażki robota są zachowane i nie blokują zadania człowieka.

Windows runtime: NOT RUN. Nie ma dostępu do czystego komputera Windows. Paczkę należy rozpakować i przetestować niezależnie od środowiska autora. Dostępny jest tylko smoke macOS i inspekcja formatu PE/ASAR.

Hosting jury: NOT READY. distribution.json ma pusty origin HTTPS. Przygotowano Dockerfile, trwały magazyn i instrukcję, ale nie wykonano wdrożenia ani sprawdzenia z innego urządzenia. Docker Desktop został następnie uruchomiony. Obraz produkcyjny zbudowano i sprawdzono lokalnie: health HTTP 200, brak .env i danych autora, proces bez uprawnień root oraz sesja zachowana po restarcie z zamontowanym wolumenem. Publiczny HTTPS nadal nie jest wdrożony.

Pilotaż: NOT RUN. Nie uczestniczyły osoby testujące. Nie podano czasów użytkowników, ich liczby ani wzrostu wiedzy. Cel 30-45 minut pozostaje założeniem produktu.

Bezpieczeństwo zależności: npm audit --omit=dev wykazał 0 podatności produkcyjnych. Po celowanej aktualizacji minimatch w zgodnych wywołaniach API liczba moderate spadła z 25 do 15. Pozostałe dotyczą starszego API minimatch w narzędziach developerskich; zmiana ich interfejsu wymaga migracji zależnych narzędzi. Lint, build i pakowanie po aktualizacji przeszły. Zależności te nie są pakowane do desktopu.

Formalności: nie uzupełniono danych i podpisów autora, sprawdzono oficjalną stronę i treść regulaminu 3 listopada 2026; lokalne pobranie oryginałów zwróciło HTTP 403, nie opublikowano linku i nie wysłano zgłoszenia. Materiały mają status lokalnego pakietu do dalszego odbioru.

## Naprawy ujawnione podczas testów

Naprawiono eksport fixture parsera, niezgodność runtime Ajv z CSP, eksporty statycznych walidatorów ESM, pozycję Piko przed lotem, pomijanie animacji, odblokowania po powtórce i nadmiarowe zależności pierwszego ASAR. Kontrola listy pól danych publicznych została rozszerzona o opcjonalne vertexLock z dalszych misji. Wyniki PASS dotyczą powtórzeń po tych naprawach.

## Reprodukcja i dalszy odbiór

Uruchom npm ci; npm run typecheck; npm run lint; npm test; npm run test:content; npm run test:e2e; npm run build; npm run desktop:dist; npm run release:check. Test AI live jest osobny i może zużywać płatne API. Raporty maszynowe w checks/ podają rzeczywiste wyniki, a IMPLEMENTATION_STATUS.md określa końcowy stan etapów.

Przeczytaj zapisane odpowiedzi live według rubryk i zatwierdź jakość dialogu. Następnie przetestuj publiczny endpoint na urządzeniu odbiorcy, odbierz Windows i pilotaż. Nie powtarzaj płatnych prób bez konkretnego powodu. Nie łącz sukcesu robota z samodzielnym wynikiem gracza.

## Dalszy odbiór

Runner live obsługuje filtr --case i limit --max-calls, sprawdza teach i transfer misji w jednej sesji, zatrzymuje test po pierwszym błędzie i zwraca niezerowy kod przy awarii. Cztery dodatkowe testy sprawdzają ten mechanizm bez płatnego API. Przygotowany ręczny workflow GitHub Actions obejmuje Windows EXE i kontener Linux; wykonanie workflow pozostaje NOT RUN, ponieważ repozytorium nie ma remote. Dostępne są protokoły pilotażu i ręcznego odbioru Windows w docs/15-pilotaz.md i docs/16-windows-odbior.md.

## Poprawki po wywołaniach prawdziwego modelu

Po krótkiej odpowiedzi bez dowodów model sugerował niekwalifikowaną regułę. Schemat przekazywany dostawcy jest teraz zawężany do bieżących faktów, dozwolonych sugestii i fazy, przy zachowaniu źródłowego kontraktu i niezależnej walidacji. Doprecyzowano rolę Piko i pojedyncze pytanie. Runner zapisuje wyniki po każdej turze i stan INTERRUPTED po zatrzymaniu, zachowując informację o możliwym wywołaniu w toku. Nowy klucz po podmianie wymagał usunięcia jednego dodatkowego znaku i przywrócenia pola AI_MODEL; sekret nie został wypisany ani zapisany w raportach.